

README.md

中国计算机学会（CCF）CSP认证 300+ 分方案

> **目标**：在 CCF CSP（Certified Software Professional）认证中取得 300 分及以上（满分 500 分），即稳定通过前三题并在第四题拿到部分分。

GitHub 新手——如何找到这里的文件？

> 如果你是第一次使用 GitHub，按以下步骤查看本仓库中的所有资料：

方法一：直接点击文件名（最简单）

在本页面（README）往上看，GitHub 页面顶部会列出仓库里的所有文件，直接点击文件名即可查看内容：

文件名	内容
README.md	你现在正在看的这个文件，CCF CSP 300+ 分备考总指南
ALGORITHMS.md	算法速查手册——遇到什么题用什么算法（含23道例题+C++代码）
PAST_EXAMS.md	近年真题汇编——7道CCF CSP真题，完整题面+C++解题代码+算法标注

方法二：如果你看到的还是旧版本（只有README），说明当前在 main 分支

GitHub 的改动存放在不同的「分支（branch）」里，正式合并前需要切换：

1. 在仓库页面，找到左上角写着 **main** 的下拉按钮（Branch 选择器）
2. 点击它，在列表中选择 **copilot/add-csp-certification-solution**
3. 页面刷新后，你就能看到 `ALGORITHMS.md` 和 `PAST_EXAMS.md` 两个新文件了

方法三：查看 Pull Request（PR）

1. 点击仓库页面顶部的 **「Pull requests」** 标签
2. 找到标题包含「CCF CSP」或「ALGORITHMS」的 PR，点击进入
3. 点击 **「Files changed」** 标签，可以看到所有新增内容
4. 点击 **「Merge pull request」** 可以将改动合并到主分支，之后在 main 分支就能直接看到

配套资料

- **[算法速查手册（ALGORITHMS.md）](ALGORITHMS.md)**：遇到什么题用什么算法，含 23 道例题与完整解析代码

- **[近年真题汇编（PAST_EXAMS.md）](PAST_EXAMS.md)**：2020 – 2022 年 CCF CSP 真题，含完整题目描述、C++ 代码与算法标注

目录

1. [考试简介](#1-考试简介)
2. [300+ 分的意义与可行性分析](#2-300-分的意义与可行性分析)
3. [知识体系与学习路线](#3-知识体系与学习路线)
4. [各题目难度与策略](#4-各题目难度与策略)
5. [学习资源推荐](#5-学习资源推荐)
6. [刷题计划（8 周）](#6-刷题计划8-周)
7. [常用算法与数据结构模板](#7-常用算法与数据结构模板)
8. [考场技巧与注意事项](#8-考场技巧与注意事项)
9. [常见失分原因及对策](#9-常见失分原因及对策)

1. 考试简介

项目	内容
主办方	中国计算机学会（CCF）
考试形式	上机编程（OJ 在线判题）
题目数量	5 道题
每题分值	100 分（含部分分）
满分	500 分
考试时长	4 小时
编程语言	C/C++、Java、Python
考试频次	每年约 3 次（3 月、9 月、12 月）
官网	https://www.ccf.org.cn/csp/

2. 300+ 分的意义与可行性分析

得分区间参考

分数段	水平描述
100 – 200	入门级，具备基本编程能力
200 – 300	初级，熟悉基本数据结构与算法
300 – 400	**中级，具备竞赛/工程基础，企业招聘关注段**
400 – 500	高级，接近算法竞赛水平（ICPC 等）

> 300 分是很多企业（尤其是互联网大厂）对应届生或实习生的参考门槛，也是研究生保研/推免时的重要加分项。

300+ 分可行性

- **第 1、2 题**（各 100 分）：难度低，以模拟、字符串处理、简单数学为主，基础扎实可**满分**。
- **第 3 题**（100 分）：中等难度，考察基本数据结构（栈、队列、哈希表）或简单图论，练习充分可**满分或接近满分**。

- **第 4 题**（100 分）：较难，动态规划、图算法、贪心等，争取拿到 **30 – 60 分**。
- **第 5 题**（100 分）：高难度，竞赛级算法，**放弃或争取部分分**。

> **保守策略**：前三题满分（300）+ 第四题部分分（30+）= **330+**

3. 知识体系与学习路线

第一阶段：编程语言基础（0 – 2 周）

必须掌握（选一门精通）：

- **C++（推荐）**：STL 容器（vector、map、set、queue、priority_queue、stack）、算法库（sort、lower_bound、upper_bound）、位运算
- **Python**：内置库（collections、heapq、bisect、itertools、functools）、列表推导式、lambda
- **Java**：Collections 框架、Arrays 工具类、PriorityQueue

重点语法：

'''

- 输入输出（cin/cout vs scanf/printf，注意速度）
- 字符串处理（substr、find、split）
- 结构体/类定义与运算符重载
- 引用与指针基础

'''

第二阶段：基础数据结构（2 – 4 周）

数据结构	CSP 考察频率	难度
----- ----- -----		
数组与前缀和		低
链表（模拟）		低
栈与队列		低
哈希表（map/set）		低
堆（优先队列）		中
二叉树与遍历		中
并查集		中
线段树 / 树状数组		高

第三阶段：基础算法（4 – 6 周）

算法	CSP 考察频率	难度
----- ----- -----		
排序与二分查找		低
贪心算法		中
递归与回溯		中
深度优先搜索（DFS）		中
广度优先搜索（BFS）		中
动态规划（DP）		高
图论（最短路、拓扑排序）		高

第四阶段：模拟与专题训练 (6-8 周)

- 刷历年 CSP 真题 (重点刷第 1-3 题, 争取第 4 题部分分)
- 分题型专项练习
- 模拟考试 (限时 4 小时)

4. 各题目难度与策略

第 1 题 (目标: 100分)

****特点:**** 简单模拟、数学运算、字符串处理

****策略:****

- 仔细阅读题意, 100% 按要求实现
- 注意边界条件 (空输入、极值)
- 不要过度优化, 暴力解通常够用
- 控制时间: ****不超过 20 分钟****

****常见题型:****

- 日期计算
- 进制转换
- 简单排序/统计
- 字符串替换/匹配

第 2 题 (目标: 100分)

****特点:**** 模拟、基本数据结构应用

****策略:****

- 抽象化问题 (通常是某种规则模拟)
- 善用 map/set 快速查找
- 注意数据范围, 避免 int 溢出 (使用 long long)
- 控制时间: ****不超过 30 分钟****

****常见题型:****

- 模拟游戏规则
- 图形输出
- 复杂统计
- 数组操作

第 3 题 (目标: 80-100分)

****特点:**** 算法与数据结构综合应用

****策略:****

- 识别题目背后的数据结构 (BFS/DFS/DP/贪心)

- 先争取通过样例，再优化时间复杂度
- 部分分策略：暴力解通常能拿 50 – 60 分
- 控制时间：**不超过 60 分钟**

****常见题型：****

- 图的遍历（BFS/DFS）
- 简单动态规划
- 贪心区间调度
- 栈/队列应用

第 4 题（目标：30 – 60分）

****特点：**** 较复杂算法，需要优化

****策略：****

- 优先理解题意，设计正确解法
- 暴力解法先保底分（50% – 60%）
- 若有时间，尝试优化至正解
- 控制时间：**剩余时间的一半**

****常见题型：****

- 复杂动态规划（背包、区间DP）
- 图论（最短路、网络流简单版）
- 数据结构优化

第 5 题（目标：0 – 30分）

****特点：**** 竞赛级难题

****策略：****

- 快速阅读，判断是否可做
- 特殊情况/边界用暴力骗分
- 剩余时间不多时，直接放弃
- 如果是熟悉的算法，快速实现核心思路

5. 学习资源推荐

刷题平台

平台	链接	推荐用途
洛谷 (Luogu)	https://www.luogu.com.cn	CSP 历年真题、算法专题
CSP 官方题库	https://csp.thussat.com	官方真题练习
LeetCode (力扣)	https://leetcode.cn	数据结构与算法专项
Codeforces	https://codeforces.com	进阶练习 (Div.3 – Div.2)
AcWing	https://www.acwing.com	算法模板与系统课程

书籍推荐

书名	作者	适合阶段
《算法竞赛入门经典》	刘汝佳	入门 – 中级
《算法（第4版）》	Sedgewick	中级
《挑战程序设计竞赛》	秋叶拓哉等	中级 – 高级
《算法导论》	CLRS	理论深度

在线课程

- **AcWing 算法基础课**：系统讲解算法模板，适合 CSP 备考
- **力扣探索**：按题型分类的系列练习
- **B站 CSP 备考视频**：搜索"CSP认证备考"有大量视频教程

6. 刷题计划（8周）

第1–2周：语言与基础

- ☐ 熟练掌握选择语言的输入输出
- ☐ 练习字符串处理题 20+ 道
- ☐ 练习简单模拟题 20+ 道
- ☐ 刷 CSP 历年第 1 题（全部）
- ☐ 目标：CSP 第 1 题稳定满分

第3–4周：数据结构

- ☐ 掌握 STL/标准库常用容器
- ☐ 练习栈、队列、哈希表题 30+ 道
- ☐ 学习并练习堆（优先队列）20+ 道
- ☐ 刷 CSP 历年第 2 题（全部）
- ☐ 目标：CSP 第 2 题稳定 80+ 分

第5–6周：基础算法

- ☐ 掌握 BFS/DFS 基本框架
- ☐ 学习简单动态规划（线性DP、背包）
- ☐ 学习贪心算法常见模型
- ☐ 刷 CSP 历年第 3 题（全部）
- ☐ 目标：CSP 第 3 题稳定 60+ 分

第7周：强化训练

- ☐ 专项练习第 3 题类型题目 20+ 道
- ☐ 开始练习第 4 题（争取暴力分）
- ☐ 学习图论基础（BFS最短路、拓扑排序）
- ☐ 目标：CSP 第 3 题稳定 80+ 分，第 4 题拿 30 分

第8周：模拟考试

- [] 每 2 天做一套历年 CSP 真题（限时 4 小时）
- [] 总结错题和薄弱知识点
- [] 查缺补漏，重点攻克卡住的题型
- [] 目标：模拟稳定 300+ 分

7. 常用算法与数据结构模板

7.1 前缀和（一维）

```
```cpp
// 构建前缀和
vector<long long> prefix(n + 1, 0);
for (int i = 1; i <= n; i++)
 prefix[i] = prefix[i - 1] + a[i];

// 区间 [l, r] 的和
long long sum = prefix[r] - prefix[l - 1];
```
```

7.2 二分查找

```
```cpp
// 查找第一个 >= target 的位置
int lo = 0, hi = n;
while (lo < hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] < target) lo = mid + 1;
 else hi = mid;
}
// lo 即为结果，等价于 lower_bound(a, a+n, target) - a
```
```

7.3 BFS（最短路/层次遍历）

```
```cpp
#include <queue>
using namespace std;

void bfs(int start, vector<vector<int>>& graph, int n) {
 vector<int> dist(n, -1);
 queue<int> q;
 dist[start] = 0;
 q.push(start);
 while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v : graph[u]) {
 if (dist[v] == -1) {
```

```

 dist[v] = dist[u] + 1;
 q.push(v);
 }
}
}
}
'''

```

### ### 7.4 DFS（图遍历/回溯）

```

'''cpp
vector<bool> visited(n, false);

void dfs(int u, vector<vector<int>>& graph) {
 visited[u] = true;
 for (int v : graph[u]) {
 if (!visited[v]) {
 dfs(v, graph);
 }
 }
}
}
'''

```

### ### 7.5 并查集

```

'''cpp
vector<int> parent, rank_;

int find(int x) {
 if (parent[x] != x)
 parent[x] = find(parent[x]); // 路径压缩
 return parent[x];
}

void unite(int x, int y) {
 int px = find(x), py = find(y);
 if (px == py) return;
 if (rank_[px] < rank_[py]) swap(px, py);
 parent[py] = px;
 if (rank_[px] == rank_[py]) rank_[px]++;
}

// 初始化
void init(int n) {
 parent.resize(n);
 rank_.assign(n, 0);
 iota(parent.begin(), parent.end(), 0);
}
'''

```



### ### 7.6 Dijkstra 最短路

```
```cpp
#include <queue>
using namespace std;
typedef pair<long long, int> pli;

vector<long long> dijkstra(int src, int n,
    vector<vector<pair<int, long long>>>& adj) {
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pli, vector<pli>, greater<pli>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

7.7 动态规划 (01背包)

```
```cpp
// 容量为 W, n 个物品, 重量 w[i], 价值 v[i]
vector<long long> dp(W + 1, 0);
for (int i = 0; i < n; i++) {
 for (int j = W; j >= w[i]; j--) {
 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
 }
}
// dp[W] 为最大价值
```

### ### 7.8 快速排序 (sort + lambda)

```
```cpp
#include <algorithm>
// 按第二关键字升序, 第一关键字降序
sort(arr.begin(), arr.end(), [](const auto& a, const auto& b) {
    if (a.first != b.first) return a.first > b.first;
    return a.second < b.second;
});
```

7.9 字符串哈希

```
```cpp
// 多项式哈希，用于字符串比较
const long long MOD = 1e9 + 7, BASE = 131;
int n = s.size();
vector<long long> h(n + 1, 0), pw(n + 1, 1);
for (int i = 0; i < n; i++) {
 h[i + 1] = (h[i] * BASE + s[i]) % MOD;
 pw[i + 1] = pw[i] * BASE % MOD;
}
// 子串 [l, r] (0-indexed) 的哈希值
auto getHash = [&](int l, int r) {
 return (h[r + 1] - h[l] * pw[r - l + 1] % MOD + MOD * 2) % MOD;
};
```
```

7.10 单调栈

```
```cpp
// 求每个元素右侧第一个比它大的元素的下标
vector<int> nextGreater(n, n); // 默认为 n（不存在）
stack<int> stk;
for (int i = 0; i < n; i++) {
 while (!stk.empty() && a[stk.top()] < a[i]) {
 nextGreater[stk.top()] = i;
 stk.pop();
 }
 stk.push(i);
}
```
```

8. 考场技巧与注意事项

时间分配建议

| 阶段 | 时间 | 任务 |
|--------|-------|-----------------------|
| 开始 | 10 分钟 | 快速浏览全部 5 题，评估难度 |
| 第 1 阶段 | 20 分钟 | 完成第 1 题 |
| 第 2 阶段 | 30 分钟 | 完成第 2 题 |
| 第 3 阶段 | 60 分钟 | 尽力完成第 3 题 |
| 第 4 阶段 | 60 分钟 | 第 4 题（暴力骗分 + 尝试正解） |
| 第 5 阶段 | 60 分钟 | 第 5 题（特殊情况骗分）或回头优化前几题 |

关键注意事项

1. ****输入输出优化**** (C++) :

```
```cpp
ios::sync_with_stdio(false);
cin.tie(nullptr);
```
```

2. ****数据类型****: 遇到大数运算务必使用 `long long`, 必要时用 `\_\_int128` (注意: `\_\_int128` 是 GCC/Clang 扩展, 不属于 C++ 标准, 在 MSVC 等编译器上不可用; CSP 评测使用 GCC, 可以放心使用, 但移植到其他平台时需改用高精度库)

3. ****边界条件****:

- 数组下标越界
- 空输入/输出
- $n=0$ 或 $n=1$ 的特殊情况

4. ****调试技巧****:

- 多打样例, 手动验证
- 使用 `cerr` 输出调试信息 (不影响正式输出)

5. ****部分分策略****:

- 先写暴力解提交
- 暴力通过后再优化
- 保存可以拿分的版本

6. ****读题****:

- 注意输入格式 (是否有多组测试数据)
- 注意输出格式 (换行、空格)
- 注意数据范围 (决定算法复杂度要求)

9. 常见失分原因及对策

| 失分原因 | 对策 |
|------------|---|
| 整数溢出 | 全程使用 `long long`, 大数相乘先转换 |
| 数组越界 | 数组开大一些 (+10), 下标使用前先检查范围 |
| 读题错误 | 拿到题先读两遍, 特别注意输出格式 |
| 算法思路错误 | 对样例手动推导, 验证思路 |
| 时间超限 (TLE) | 分析时间复杂度, $O(n^2)$ 以上要优化; 先检查是否有不必要的 I/O |
| 空间超限 (MLE) | 分析空间复杂度, 减少不必要的拷贝 |
| 精度问题 (浮点数) | 尽量用整数运算; 比较浮点数用 `eps = 1e-9` |
| 忘记初始化 | 全局变量默认初始化, 局部变量一定要显式初始化 |

总结: 300+ 分核心路径

稳固基础 (语言 + STL)

↓
第1、2题满分（200分）——练习量：50 – 100题
↓
第3题稳定拿分（60 – 100分）——练习量：BFS/DFS/DP 各20题
↓
第4题暴力骗分（30 – 60分）——理解题意，写出 $O(n^2)$ 或 $O(n^3)$ 暴力
↓
总计：290 – 360分 超过300分目标
'''

> **最重要的是：持续练习 + 总结错题 + 模拟实战。** 坚持 8 周系统训练，300+ 分完全可以实现。

本方案基于 CCF CSP 历年真题分析与算法学习路线整理，持续更新中。